

User: hi User: hi

GitHub Copilot: Optimized tool selectionHi! What can I help you with in this repo?

User: You're a professional full-stack engineer.

Your general task is to create a simple Gradio UI to be hosted in huggingface spaces which takes the information needed to make an inference with 'model/stacking_classifier.joblib' on whether the user will catch fish or not, depending on the current weather conditions and their planned mode of fishing, hours fished, etc (Required data will be followed up by agent completing task when the agent needs it).

What you're going to deliver now is NOT CODE. You will create 3 markdown files:

- context.md : describes the context of the frontend along with a short description of each component so that a new agent can immediately understand what is going on
- task-checklist.md : breaks down the general task into subtasks which can be checked off as they are done (1 per prompt)
- agent-instructions.md : gives an overview of the instructions for each task. It can be exactly this: "# Agent Instructions (Execution Contract)

Read Order (mandatory)

1. context.md
2. agent-instructions.md (this file)
3. task-checklist.md

Required Behavior For Every Run

1. Find the first incomplete task in task-checklist.md (first ` - [ ] Txx ...` item).
2. Complete only that task efficiently and professionally.
3. Keep implementation pragmatic; avoid unnecessary comments.
4. Write unit tests and/or integration tests for the implemented task.
5. Run tests and ensure they pass before finishing.
6. If tests fail and cannot be fixed safely in this run, stop and report issues to the user with exact failures and proposed options.
7. Update task-checklist.md:
 - Mark completed task as `[x]` so it is no longer incomplete.
 - If blocked, leave as `[ ]` and append `BLOCKED: <reason>` directly under the task.
 - Add a line to the Completion Log.
8. Generate the ideal prompt for the next task (the next first incomplete item).
9. In the generated next prompt, instruct the next agent to read context.md, agent-instructions.md, and task-checklist.md before coding.

Quality Standard

- Prefer small, verifiable increments.
- Do not change scope beyond the active task.

- Do not defer testing.
- Any operation that can affect cost or cloud resources must include safe defaults.

Required Response Template

Use this structure in your final message each run:

1. Task completed: `Txx <title>`
2. What changed: short implementation summary
3. Tests added: list
4. Test results: pass/fail summary
5. Checklist update: task status + completion log line
6. Next prompt: ready-to-copy prompt for the next agent

Next Prompt Template

Use this template to generate the next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

DO THIS ALL IN THE /Frontend directory

GitHub Copilot: Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Created the three markdown files in Frontend:

- context.md
- task-checklist.md
- agent-instructions.md

Want me to adjust the task list or component descriptions before the next agent starts?

Made changes.

User: This is more of what the task-checklist should be like as an example:

TPU Tool Strict Task Checklist

Rules

1. Execute only the first incomplete task (top to bottom).
2. Do not skip, reorder, or bundle future tasks.
3. Keep implementations utilitarian and production-safe.
4. Every task must include tests and passing evidence.
5. Use context.md and agent-instructions.md before writing code.

Tasks

- [x] T01 Bootstrap CLI project and test harness
 - Deliverables: project scaffold, CLI endpoint, dependency config, test command.
 - Exit criteria: CLI help command works; tests run from one command.
 - Required tests: CLI smoke test; config bootstrap unit test.
- [x] T02 Implement configuration and settings model
 - Deliverables: typed settings loader for project/env/runtime limits.
 - Exit criteria: invalid config fails fast with clear errors.
 - Required tests: valid/invalid config unit tests.
- [x] T03 Implement authentication and account validation command
 - Deliverables: `auth check` command validating active account, project, and required APIs.
 - Exit criteria: command provides pass/fail output with actionable remediation.
 - Required tests: mocked auth success/failure unit tests.
- [x] T04 Implement metadata and storage repositories
 - Deliverables: Firestore repository (jobs/sessions/settings) and GCS artifact repository.
 - Exit criteria: create/read/update operations for core records are stable.
 - Required tests: repository unit tests with mocks/fakes.
- [x] T05 Implement pre-run cost gate
 - Deliverables: estimator for projected run cost and monthly soft-cap gate.
 - Exit criteria: over-cap submissions are blocked before resource creation.
 - Required tests: estimator unit tests; gate pass/fail tests.
- [x] T06 Implement global concurrency lock
 - Deliverables: single-active-workload lock using shared cloud state.
 - Exit criteria: second concurrent run is blocked deterministically.
 - Required tests: lock acquire/release tests; contention test.

- [x] T07 Implement TPU VM lifecycle manager
 - Deliverables: create/start/stop/delete TPU VM functions with retries and idempotency.
 - Exit criteria: repeated cleanup calls do not fail unpredictably.
 - Required tests: lifecycle unit tests with API mocks.
- [x] T08 Implement batch script execution pipeline
 - Deliverables: `submit script` flow (upload inputs, execute, collect logs/artifacts, finalize status).
 - Exit criteria: script run transitions through correct states and stores artifacts.
 - Required tests: integration test with mocked cloud services.
- [x] T09 Implement notebook execution pipeline
 - Deliverables: `submit notebook` flow with headless execution and executed notebook export.
 - Exit criteria: executed notebook persisted; failures capture traceback summary.
 - Required tests: notebook flow integration test.
- [x] T10 Implement job control commands
 - Deliverables: `status`, `logs`, and `cancel` commands.
 - Exit criteria: cancel always triggers cleanup and final status update.
 - Required tests: command handler tests and cancel-path integration test.
- [x] T11 Enforce runtime timeout and guaranteed cleanup
 - Deliverables: max-runtime hard stop and finalizer behavior for success/failure/cancel/timeout.
 - Exit criteria: no leaked TPU resources after abnormal exits.
 - Required tests: timeout and cleanup integration tests.
- [x] T12 Implement monthly usage reporting
 - Deliverables: `usage month` summary from stored run/session ledger.
 - Exit criteria: per-user and total monthly estimates are displayed clearly.
 - Required tests: ledger aggregation unit tests.
- [x] T13 Implement interactive TPU session mode
 - Deliverables: `session start`, `session list`, `session stop` with mandatory TTL.
 - Exit criteria: connection details provided; auto-expiry enforced.
 - Required tests: session command tests and TTL expiration test.
- [x] T14 Implement janitor cleanup automation
 - Deliverables: scheduled cleanup for orphaned VMs and stale sessions.
 - Exit criteria: stale resources are detected and terminated safely.
 - Required tests: janitor decision logic unit tests; one integration test.
- [x] T15 Apply security hardening and authorization controls
 - Deliverables: allowlist/group checks, least-privilege validation, audit logging for cost-incurring actions.

- Exit criteria: unauthorized callers are blocked before mutation actions.
- Required tests: authorization unit tests; audit event assertions.
- [x] T16 Run end-to-end validation and finalize docs
 - Deliverables: end-to-end runbook, setup notes, known limitations, operational checklist.
 - Exit criteria: script and notebook flows validated; budget/cleanup safeguards validated.
 - Required tests: full integration test pass report and manual validation notes.

Completion Log

- Add one line per finished task: `YYYY-MM-DD | Task ID | Agent | Result | Test Summary`
- 2026-04-21 | T01 | GitHub Copilot | Completed | 3 passed via `python -m pytest` (CLI smoke + config bootstrap)
- 2026-04-21 | T02 | GitHub Copilot | Completed | 7 passed via python -m pytest (settings valid and invalid unit tests)
- 2026-04-21 | T03 | GitHub Copilot | Completed | 9 passed via python -m pytest (mocked auth check success/failure)
- 2026-04-21 | T04 | GitHub Copilot | Completed | 13 passed via python -m pytest (repository unit tests with fakes)
- 2026-04-21 | T05 | GitHub Copilot | Completed | 18 passed via python -m pytest (estimator and gate pass/fail tests)
- 2026-04-21 | T06 | GitHub Copilot | Completed | 20 passed via python -m pytest (lock acquire/release and contention tests)
- 2026-04-21 | T07 | GitHub Copilot | Completed | 24 passed via python -m pytest (lifecycle retries and idempotent cleanup tests)
- 2026-04-21 | T08 | GitHub Copilot | Completed | 26 passed via python -m pytest (mocked-cloud batch script integration tests)
- 2026-04-21 | T09 | GitHub Copilot | Completed | 27 passed via python -m pytest (mocked notebook integration test)
- 2026-04-21 | T10 | GitHub Copilot | Completed | 41 passed via python -m pytest (job control handler tests + cancel-path integration test)
- 2026-04-21 | T11 | GitHub Copilot | Completed | 49 passed via python -m pytest (runtime timeout + guaranteed cleanup integration tests for script and notebook pipelines)
- 2026-04-21 | T12 | GitHub Copilot | Completed | 58 passed via python -m pytest (monthly usage aggregation, rendering, and CLI handler tests)
- 2026-04-21 | T13 | GitHub Copilot | Completed | 75 passed via python -m pytest (session start/list/stop + TTL expiration tests)
- 2026-04-21 | T14 | GitHub Copilot | Completed | 86 passed via python -m pytest (janitor decision + CLI integration tests)
- 2026-04-21 | T15 | GitHub Copilot | Completed | 101 passed via python -m pytest (authorization + audit logging tests)
- 2026-04-21 | T16 | GitHub Copilot | Completed | 103 passed via python -m pytest (end-to-end integration + docs)

GitHub Copilot: Generating patch (57 lines) in
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

...

...

Updated the checklist to match your strict format in task-checklist.md.
Want me to align any wording in the tasks or add more detailed exit criteria?

Made changes.

User: alright, now write the prompt to begin implementation

GitHub Copilot: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Optimized tool selectionHi! What can I help you with in this repo?

User: You're a professional full-stack engineer.

Your general task is to create a simple Gradio UI to be hosted in huggingface spaces which takes the information needed to make an inference with 'model/stacking_classifier.joblib' on whether the user will catch fish or not, depending on the current weather conditions and their planned mode of fishing, hours fished, etc (Required data will be followed up by agent completing task when the agent needs it).

What you're going to deliver now is NOT CODE. You will create 3 markdown files:

- context.md : describes the context of the frontend along with a short description of each component so that a new agent can immediately understand what is going on
- task-checklist.md : breaks down the general task into subtasks which can be checked off as they are done (1 per prompt)
- agent-instructions.md : gives an overview of the instructions for each task. It can be exactly this: "# Agent Instructions (Execution Contract)

Read Order (mandatory)

1. context.md
2. agent-instructions.md (this file)
3. task-checklist.md

Required Behavior For Every Run

1. Find the first incomplete task in task-checklist.md (first `-[ ] Txx ...` item).
2. Complete only that task efficiently and professionally.
3. Keep implementation pragmatic; avoid unnecessary comments.
4. Write unit tests and/or integration tests for the implemented task.
5. Run tests and ensure they pass before finishing.
6. If tests fail and cannot be fixed safely in this run, stop and report issues to the user with exact failures and proposed options.
7. Update task-checklist.md:
 - Mark completed task as `[x]` so it is no longer incomplete.
 - If blocked, leave as `[ ]` and append `BLOCKED: <reason>` directly under the task.
 - Add a line to the Completion Log.
8. Generate the ideal prompt for the next task (the next first incomplete item).
9. In the generated next prompt, instruct the next agent to read context.md, agent-instructions.md, and task-checklist.md before coding.

Quality Standard

- Prefer small, verifiable increments.
- Do not change scope beyond the active task.
- Do not defer testing.
- Any operation that can affect cost or cloud resources must include safe defaults.

Required Response Template

Use this structure in your final message each run:

1. Task completed: `Txx <title>`
2. What changed: short implementation summary
3. Tests added: list
4. Test results: pass/fail summary
5. Checklist update: task status + completion log line
6. Next prompt: ready-to-copy prompt for the next agent

Next Prompt Template

Use this template to generate the next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

"

DO THIS ALL IN THE /Frontend directory

GitHub Copilot: Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Created the three markdown files in Frontend:

- context.md
- task-checklist.md
- agent-instructions.md

Want me to adjust the task list or component descriptions before the next agent starts?

Made changes.

User: This is more of what the task-checklist should be like as an example:

TPU Tool Strict Task Checklist

Rules

1. Execute only the first incomplete task (top to bottom).
2. Do not skip, reorder, or bundle future tasks.
3. Keep implementations utilitarian and production-safe.
4. Every task must include tests and passing evidence.
5. Use context.md and agent-instructions.md before writing code.

Tasks

- [x] T01 Bootstrap CLI project and test harness
 - Deliverables: project scaffold, CLI entrypoint, dependency config, test command.
 - Exit criteria: CLI help command works; tests run from one command.
 - Required tests: CLI smoke test; config bootstrap unit test.
- [x] T02 Implement configuration and settings model
 - Deliverables: typed settings loader for project/env/runtime limits.
 - Exit criteria: invalid config fails fast with clear errors.
 - Required tests: valid/invalid config unit tests.

- [x] T03 Implement authentication and account validation command
 - Deliverables: `auth check` command validating active account, project, and required APIs.
 - Exit criteria: command provides pass/fail output with actionable remediation.
 - Required tests: mocked auth success/failure unit tests.
- [x] T04 Implement metadata and storage repositories
 - Deliverables: Firestore repository (jobs/sessions/settings) and GCS artifact repository.
 - Exit criteria: create/read/update operations for core records are stable.
 - Required tests: repository unit tests with mocks/fakes.
- [x] T05 Implement pre-run cost gate
 - Deliverables: estimator for projected run cost and monthly soft-cap gate.
 - Exit criteria: over-cap submissions are blocked before resource creation.
 - Required tests: estimator unit tests; gate pass/fail tests.
- [x] T06 Implement global concurrency lock
 - Deliverables: single-active-workload lock using shared cloud state.
 - Exit criteria: second concurrent run is blocked deterministically.
 - Required tests: lock acquire/release tests; contention test.
- [x] T07 Implement TPU VM lifecycle manager
 - Deliverables: create/start/stop/delete TPU VM functions with retries and idempotency.
 - Exit criteria: repeated cleanup calls do not fail unpredictably.
 - Required tests: lifecycle unit tests with API mocks.
- [x] T08 Implement batch script execution pipeline
 - Deliverables: `submit script` flow (upload inputs, execute, collect logs/artifacts, finalize status).
 - Exit criteria: script run transitions through correct states and stores artifacts.
 - Required tests: integration test with mocked cloud services.
- [x] T09 Implement notebook execution pipeline
 - Deliverables: `submit notebook` flow with headless execution and executed notebook export.
 - Exit criteria: executed notebook persisted; failures capture traceback summary.
 - Required tests: notebook flow integration test.
- [x] T10 Implement job control commands
 - Deliverables: `status`, `logs`, and `cancel` commands.
 - Exit criteria: cancel always triggers cleanup and final status update.
 - Required tests: command handler tests and cancel-path integration test.
- [x] T11 Enforce runtime timeout and guaranteed cleanup
 - Deliverables: max-runtime hard stop and finalizer behavior for success/failure/cancel/timeout.
 - Exit criteria: no leaked TPU resources after abnormal exits.

- Required tests: timeout and cleanup integration tests.
- [x] T12 Implement monthly usage reporting
 - Deliverables: `usage month` summary from stored run/session ledger.
 - Exit criteria: per-user and total monthly estimates are displayed clearly.
 - Required tests: ledger aggregation unit tests.
- [x] T13 Implement interactive TPU session mode
 - Deliverables: `session start`, `session list`, `session stop` with mandatory TTL.
 - Exit criteria: connection details provided; auto-expiry enforced.
 - Required tests: session command tests and TTL expiration test.
- [x] T14 Implement janitor cleanup automation
 - Deliverables: scheduled cleanup for orphaned VMs and stale sessions.
 - Exit criteria: stale resources are detected and terminated safely.
 - Required tests: janitor decision logic unit tests; one integration test.
- [x] T15 Apply security hardening and authorization controls
 - Deliverables: allowlist/group checks, least-privilege validation, audit logging for cost-incurring actions.
 - Exit criteria: unauthorized callers are blocked before mutation actions.
 - Required tests: authorization unit tests; audit event assertions.
- [x] T16 Run end-to-end validation and finalize docs
 - Deliverables: end-to-end runbook, setup notes, known limitations, operational checklist.
 - Exit criteria: script and notebook flows validated; budget/cleanup safeguards validated.
 - Required tests: full integration test pass report and manual validation notes.

Completion Log

- Add one line per finished task: `YYYY-MM-DD | Task ID | Agent | Result | Test Summary`
- 2026-04-21 | T01 | GitHub Copilot | Completed | 3 passed via `python -m pytest` (CLI smoke + config bootstrap)
- 2026-04-21 | T02 | GitHub Copilot | Completed | 7 passed via python -m pytest (settings valid and invalid unit tests)
- 2026-04-21 | T03 | GitHub Copilot | Completed | 9 passed via python -m pytest (mocked auth check success/failure)
- 2026-04-21 | T04 | GitHub Copilot | Completed | 13 passed via python -m pytest (repository unit tests with fakes)
- 2026-04-21 | T05 | GitHub Copilot | Completed | 18 passed via python -m pytest (estimator and gate pass/fail tests)
- 2026-04-21 | T06 | GitHub Copilot | Completed | 20 passed via python -m pytest (lock acquire/release and contention tests)
- 2026-04-21 | T07 | GitHub Copilot | Completed | 24 passed via python -m pytest (lifecycle retries and idempotent cleanup tests)

- 2026-04-21 | T08 | GitHub Copilot | Completed | 26 passed via python -m pytest (mocked-cloud batch script integration tests)
- 2026-04-21 | T09 | GitHub Copilot | Completed | 27 passed via python -m pytest (mocked notebook integration test)
- 2026-04-21 | T10 | GitHub Copilot | Completed | 41 passed via python -m pytest (job control handler tests + cancel-path integration test)
- 2026-04-21 | T11 | GitHub Copilot | Completed | 49 passed via python -m pytest (runtime timeout + guaranteed cleanup integration tests for script and notebook pipelines)
- 2026-04-21 | T12 | GitHub Copilot | Completed | 58 passed via python -m pytest (monthly usage aggregation, rendering, and CLI handler tests)
- 2026-04-21 | T13 | GitHub Copilot | Completed | 75 passed via python -m pytest (session start/list/stop + TTL expiration tests)
- 2026-04-21 | T14 | GitHub Copilot | Completed | 86 passed via python -m pytest (janitor decision + CLI integration tests)
- 2026-04-21 | T15 | GitHub Copilot | Completed | 101 passed via python -m pytest (authorization + audit logging tests)
- 2026-04-21 | T16 | GitHub Copilot | Completed | 103 passed via python -m pytest (end-to-end integration + docs)

GitHub Copilot: Generating patch (57 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)

...

...

Updated the checklist to match your strict format in task-checklist.md.

Want me to align any wording in the tasks or add more detailed exit criteria?

Made changes.

User: alright, now write the prompt to begin implementation

GitHub Copilot: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Searched for regex `feature|features|X\_train|input|columns|predictors|model`
(`\*\*/\*.py,ipynb,md`), 200 results

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend)]

Configuring a Python Environment

Completed with input: {

```
"resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"
}
```

Creating a Virtual Environment

Completed with input: {

```
"resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"
}
```

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor",
"codeSnippet": "import pandas as pd\nfrom pathlib import Path\npath =\nPath('data/processed/df_tripdata_engineered_with_total.csv')\nprint(path.exists())\nndf =\npd.read_csv(path)\nprint(df.shape)\nprint(df.columns.tolist())\n",
"workingDirectory": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"
}
```

Ran `pylanceRunCodeSnippet`

Completed with input: {

```
"workspaceRoot": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor",
```

```

"codeSnippet": "import pandas as pd\nfrom pathlib import Path\npath =  
Path('data/processed/df_tripdata_engineered_with_total.csv')\nndf =  
pd.read_csv(path)\nfeature_cols = [c for c in df.columns if c not in {'has_catch','total_catch'}]\nX  
= df[feature_cols]\nprint('features', len(feature_cols))\nprint(X.dtypes.value_counts())\n# Show  
non-numeric columns\nnon_numeric =  
X.select_dtypes(exclude=['number']).columns.tolist()\nprint('non_numeric', non_numeric)\n# For  
each column, get min/max and unique count for non-numeric\nsummary = []\nfor col in  
feature_cols:\n    if pd.api.types.is_numeric_dtype(X[col]):\n        summary.append((col,  
'numeric', float(X[col].min()), float(X[col].max()), None))\n    else:\n        summary.append((col,  
'categorical', None, None, int(X[col].nunique())))\nprint(summary)\n",  
    "workingDirectory": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"  
}

```

Ran `pylanceRunCodeSnippet`

Completed with input: {

```

    "workspaceRoot": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor",  
    "codeSnippet": "import pandas as pd\nfrom pathlib import Path\npath =  
Path('data/processed/df_tripdata_engineered_with_total.csv')\nndf =  
pd.read_csv(path)\nprint(df['KOD_we'].dtype)\nprint(df['KOD_we'].head().tolist())\n",  
    "workingDirectory": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"  
}

```

Ran `pylanceRunCodeSnippet`

Completed with input: {

```

    "workspaceRoot": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor",  
    "codeSnippet": "import pandas as pd\nfrom pathlib import Path\npath =  
Path('data/processed/df_tripdata_engineered_with_total.csv')\nndf =  
pd.read_csv(path)\nfeature_cols = [c for c in df.columns if c not in {'has_catch','total_catch'}]\nX  
= df[feature_cols]\n# Build schema entries\nentries = []\nfor col in feature_cols:\n    series =  
X[col]\n    if series.dtype == 'bool':\n        col_type = 'bool'\n        default =  
bool(series.mode(dropna=True).iloc[0])\n        min_val = 0\n        max_val = 1\n    elif  
pd.api.types.is_integer_dtype(series):\n        col_type = 'int'\n        default =  
int(series.median())\n        min_val = int(series.min())\n        max_val = int(series.max())\n    else:\n        col_type = 'float'\n        default = float(series.median())\n        min_val =  
float(series.min())\n        max_val = float(series.max())\n    entries.append((col, col_type,  
min_val, max_val, default))\nfor entry in entries:\n    print(entry)\n",  
    "workingDirectory": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"  
}

```

Created [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/schema.py)

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_schema.py)

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)

Generating patch (269 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)

...

...

Generating patch (10 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)

...

...

Ran terminal command:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests

1. Task completed: T01 Define required input schema for inference
2. What changed: Defined a concrete input schema with defaults and validation helpers in schema.py, documented the full feature list in context.md, and added schema validation tests in test_schema.py.
3. Tests added: Schema validation unit tests in test_schema.py.
4. Test results: Passed —
`c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests`
5. Checklist update: Marked T01 complete and logged completion in task-checklist.md.
6. Next prompt:
"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: "You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Created [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/app.py)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_ui.py)]

Ran terminal command:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Completed with input:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Installing gradio in Python 3.12.3 (.venv)

Completed with input: {

 "packageList": [

 "gradio"

],

 "resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"

}

Ran terminal command:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Generating patch (11 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

...

...

1. Task completed: T02 Implement Gradio UI layout and input widgets
2. What changed: Added a Gradio Blocks UI with schema-backed inputs and a placeholder prediction flow in app.py, plus a UI smoke test in test_ui.py.
3. Tests added: UI smoke tests in test_ui.py.
4. Test results: Passed —
`c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests` (ResourceWarnings about unclosed event loop and SSL socket emitted during Gradio import/runtime).
5. Checklist update: Marked T02 complete and logged completion in task-checklist.md.
6. Next prompt:
"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Generating patch (24 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/app.py)]

Created [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/predictor.py)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_predictor.py)

...

...

Ran terminal command:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests

Completed with input:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests

Installing joblib in Python 3.12.3 (.venv)

Completed with input: {

```
"packageList": [  
  "joblib"  
],  
"resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"  
}
```

Ran terminal command:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests

Generating patch (12 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)

...

...

1. Task completed: T03 Implement prediction handler and model loading
2. What changed: Added a cached model loader and prediction handler in predictor.py, wired the UI to call it in app.py, and added a mocked prediction unit test in test_predictor.py.
3. Tests added: Prediction unit test in test_predictor.py.
4. Test results: Passed —
`c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests` (ResourceWarnings about unclosed event loop emitted during
Gradio import/runtime).
5. Checklist update: Marked T03 complete and logged completion in task-checklist.md.

6. Next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/schema.py)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/app.py)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/predictor.py)]

Generating patch (16 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/predictor.py)]

...

...

Generating patch (35 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/app.py)]

...

...

Generating patch (22 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_predictor.py)

...

...

Ran terminal command:

c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests

Generating patch (11 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)

...

...

1. Task completed: T04 Add validation and user-friendly output formatting
2. What changed: Enforced schema validation before inference in predictor.py and added formatted, user-friendly output plus validation error messaging in app.py.
3. Tests added: Validation unit test in test_predictor.py.
4. Test results: Passed —
`c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests` (ResourceWarnings about unclosed event loop/SSL socket emitted
during Gradio import/runtime).
5. Checklist update: Marked T04 complete and logged completion in task-checklist.md.
6. Next prompt:
"You are continuing the Frontend implementation. First read context.md, then
agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not
work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run
them, and ensure they pass. If tests fail, stop and report issues with options. Update
task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to
Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)
```

Read

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)
```

Read

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)
```

Generating patch (15 lines) in

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_ui.py)
```

Created

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_integration.py)
```

...

...

Ran terminal command:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests
```

Completed with input:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests
```

Installing scikit-learn in Python 3.12.3 (.venv)

Completed with input: {

```
  "packageList": [
    "scikit-learn"
  ],
  "resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"
}
```

Ran terminal command:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests
```

Completed with input:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests
```

Installing lightgbm in Python 3.12.3 (.venv)

Completed with input: {

```
"packageList": [
  "lightgbm"
],
"resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"
}
```

Ran terminal command:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests
```

Completed with input:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests
```

Installing xgboost in Python 3.12.3 (.venv)

Completed with input: {

```
"packageList": [
  "xgboost"
],
"resourcePath": "c:\\Users\\ross9\\Documents\\GitHub\\fishing-predictor"
}
```

Ran terminal command:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest
discover -s Frontend/tests
```

Generating patch (11 lines) in

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)
```

...

...

1. Task completed: T05 Add tests for inference and UI behavior

2. What changed: Added an inference integration test in test_integration.py and expanded UI behavior tests to verify input labels match the schema in test_ui.py.

3. Tests added: Inference integration test in test_integration.py; UI behavior test in test_ui.py.

4. Test results: Passed —

`c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend/tests` (warnings about unclosed event loop/SSL sockets and sklearn version mismatch emitted during model load; tests still passed).

5. Checklist update: Marked T05 complete and logged completion in task-checklist.md.

6. Next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend)]

Created [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/README.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/requirements.txt)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/test_entrypoint.py)]

Ran terminal command:

```
c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend
```

Generating patch (11 lines) in

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)
```

...

...

1. Task completed: T06 Add Space entry point and run instructions
2. What changed: Added a simple Space entrypoint test in test_entrypoint.py, created run instructions in README.md, and captured dependencies in requirements.txt.
3. Tests added: Entrypoint import smoke test in test_entrypoint.py.
4. Test results: Passed —
`c:/Users/ross9/Documents/GitHub/fishing-predictor/.venv/Scripts/python.exe -m unittest discover -s Frontend` (ResourceWarnings about unclosed event loop emitted during Gradio import/runtime).

5. Checklist update: Marked T06 complete and logged completion in task-checklist.md.

6. Next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md#1-1), lines 1 to 200
```

Read

```
[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)
```

Read

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)`

All tasks in task-checklist.md are already complete. There is no first incomplete task to execute.

If you want to proceed, tell me what you'd like to do next.

You will plan how to integrate live weather auto-fill for the nyc area only (this is the only location the project is for).

You must fill all inputs except for FFDAYS2, HRSF, MODE_F, CNTRBTRS. IMP_REC will always be 0, so don't include it in the form (this is the imputed data flag). You will complete all the engineered features the same way as done in the rest of the codebase based on the forecast for the upcoming HRSF (hours fished) value. The HRSF input is the user's expectation of how long they will be fishing for, so don't look backwards in weather data for feature engineering, but forwards.

Again, DON'T CODE YET. You will simply append the necessary tasks to implement this feature in task-checklist.md the same way we have done so far.

Read

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)`

Generating patch (33 lines) in

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)`

...

...

Added the NYC live weather auto-fill plan tasks to task-checklist.md, starting at T07.

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Generating patch (30 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

...

...

Generating patch (14 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

...

...

Ran terminal command: python -m unittest discover -s Frontend/tests

Completed with input: python -m unittest discover -s Frontend/tests

Ran terminal command: & .venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Task completed: T07 Plan NYC live weather auto-fill inputs

What changed: Added the NYC auto-fill plan detailing user-supplied vs auto-filled inputs, plus IMP_REC removal guidance in context.md. Marked T07 complete and logged the run in task-checklist.md.

Tests added: None (documentation-only task).

Test results: `python -m unittest discover -s Frontend/tests` (via .venv Python) — pass with warnings about unclosed event loops and sklearn model version mismatch.

Checklist update: T07 marked [x]; completion log appended in task-checklist.md.

Next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/app.py)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/predictor.py)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/schema.py)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_predictor.py)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_integration.py)]

Searched for files matching `\*\*/\*forecast\*.py`, no matches

Searched for files matching `\*\*/\*weather\*.py`, no matches

Searched for files matching `\*\*/Frontend/\*\*/\*.\*py`, 8 matches

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/requirements.txt)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_schema.py)]

Read [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_ui.py)]

Created [(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/forecast.py)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_forecast.py)]

Ran terminal command: & .venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Generating patch (12 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

...

...

Task completed: T08 Implement NYC forecast fetcher

What changed: Added a configurable NYC forecast client with provider selection, API key handling, and normalized hourly output in forecast.py. Added mocked success/error-path unit tests in test_forecast.py.

Tests added: test_forecast.py

Test results: `python -m unittest discover -s Frontend/tests` (via .venv Python) — pass with existing ResourceWarning about unclosed event loops and sklearn version warnings.

Checklist update: T08 marked [x]; completion log appended in task-checklist.md.

Next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md#1-1), lines 1 to 200]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/feature_engineering.py)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_feature_engineering.py)]

Ran terminal command: & .venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Completed with input: & .venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Generating patch (6 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_feature_engineering.py)]

...

...

Ran terminal command: & .venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Generating patch (12 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)

...

...

1. Task completed: T09 Build forward-looking feature engineering pipeline
2. What changed: Added forecast-driven feature generation with forward-looking windowing and schema clamping in feature_engineering.py; added deterministic unit coverage in test_feature_engineering.py.
3. Tests added: test_feature_engineering.py
4. Test results: `python -m unittest discover -s Frontend/tests` (via .venv Python) — pass with existing ResourceWarning and sklearn version warnings.
5. Checklist update: T09 marked [x]; completion log appended in task-checklist.md.
6. Next prompt:
"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md#1-1), lines 1 to 200

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)

Read

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)`

Generating patch (103 lines) in

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/app.py)`

...

...

Generating patch (45 lines) in

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_ui.py)`

...

...

Ran terminal command: `& .venv/Scripts/python.exe -m unittest discover -s Frontend/tests`

Generating patch (12 lines) in

`[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)`

...

...

1. Task completed: T10 Integrate auto-fill into UI flow
2. What changed: Wired NYC auto-fill into the UI with forecast-driven updates, removed IMP_REC from the form, and ensured only FFDAYS2, HRSF, MODE_F, CNTRBTRS remain interactive in app.py. Updated UI tests and added auto-fill behavior coverage in test_ui.py.
3. Tests added: test_ui.py
4. Test results: ``python -m unittest discover -s Frontend/tests`` (via .venv Python) — pass with existing ResourceWarning and sklearn version warnings.
5. Checklist update: T10 marked [x]; completion log appended in task-checklist.md.
6. Next prompt:
"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update

task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

User: You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task.

GitHub Copilot: Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/context.md#1-1), lines 1 to 200]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

Read

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/agent-instructions.md)]

Created

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/tests/test_autofill_integration.py)]

Ran terminal command: & .venv/Scripts/python.exe -m unittest discover -s Frontend/tests

Generating patch (12 lines) in

[(file:///c%3A/Users/ross9/Documents/GitHub/fishing-predictor/Frontend/task-checklist.md)]

...

...

1. Task completed: T11 Add end-to-end auto-fill inference test
2. What changed: Added an end-to-end auto-fill inference test that runs forecast-driven feature generation into prediction with a mocked model in test_autofill_integration.py.
3. Tests added: test_autofill_integration.py
4. Test results: `python -m unittest discover -s Frontend/tests` (via .venv Python) — pass with existing ResourceWarning and sklearn version warnings.
5. Checklist update: T11 marked [x]; completion log appended in task-checklist.md.
6. Next prompt:

"You are continuing the Frontend implementation. First read context.md, then agent-instructions.md, then task-checklist.md. Execute only the first incomplete task and do not work ahead. Implement it efficiently (no unnecessary comments), add unit/integration tests, run them, and ensure they pass. If tests fail, stop and report issues with options. Update task-checklist.md (mark completed task as [x], or add BLOCKED note if needed), append to Completion Log, and then generate the next prompt for the following task."

Made changes.

No, I'm sorry. I failed to respect professional boundaries when it comes to topics, and it was definitely a learning experience that just because something was a point of free discussion somewhere else I worked doesn't mean it is everywhere. I do know it's a very touchy thing especially for those in the position of saving lives, and absolutely so for a program that's recorded. I sincerely apologize not just for bringing it up, but as well for putting you in a spot that you had to stop me. It will never happen again. I do ask that if it is possible, could you cut that minute from the recording? I acknowledge the mistake I made but if you could do me the favor of ensuring it doesn't come back to bite me I would appreciate it. I also understand if it is just out of your control what stays in and out. I was only hoping to discuss the topic of using reinforcement learning, weaponry or anything of harm to human life should never come up. Again, I can not express enough my regret for this kind of interruption to your class. I hope you do not think I find the effects of this kind of thing exciting, or that I am insensitive to the loss of life going forward.